

Gegenseitige Begutachtung von Arbeiten mit PEERREVIEW

Mit PEERREVIEW soll ein Web-basiertes System entstehen, das die gegenseitige Begutachtung von Arbeiten – etwa Seminar- oder Projektarbeiten – durch Studierende oder Mitarbeitende einer Organisation systematisch unterstützt. PEERREVIEW übernimmt die Einreichung von Arbeiten, die Zuweisung von Gutachtern, die Erfassung von Bewertungen und Kommentaren sowie die Aggregation und Rückgabe der Gutachten an die Einreichenden.

PEERREVIEW besitzt eine Web-Oberfläche, über die Einreichungen verwaltet, Gutachten erstellt und Ergebnisse eingesehen werden können. Zudem soll PEERREVIEW eine API-Schnittstelle besitzen, die seine Dienste auch anderen Systemen (z. B. Lernmanagementsystemen) zur Verfügung stellt.

Denkbare Funktionen sind u. a.: Einreichung von Dokumenten (PDF, ZIP, ...), regelbasierte oder manuelle Zuweisung von Gutachtern, strukturierte Bewertungsformulare, anonymisierte Begutachtung, Fristen- und Erinnerungsverwaltung sowie statistische Auswertungen. Durch eine Plugin-Architektur sollen unterschiedliche Begutachtungsworkflows (einfach-blind, doppelt-blind, open review) unterstützt werden.

Wir wollen in der Veranstaltung das Grundgerüst von PEERREVIEW als komponentenbasiertes Microservice-System konzipieren und realisieren und dabei einen starken Fokus auf die Architektur legen: Wie können Workflow-Zustände zuverlässig verwaltet, Benachrichtigungen ereignisgesteuert ausgelöst und die Komponenten geeignet skaliert werden?

Als Technik zur Realisierung der Komponenten können Sie eine Plattform (Java, C#, Ruby, Go, Python ...) Ihrer Wahl einsetzen. Die Repräsentation der auszutauschenden Daten soll in JSON [1] erfolgen (Media-Type: `application/json`).

1. Grobentwurf

Gemeinsam legen wir den Funktionsumfang des gewünschten Systems fest und bestimmen, welche Funktionalität unterstützt werden soll. Hierbei lassen wir uns vom Gedanken des Minimal Viable Products (MVP) leiten. Weitere Funktionalität soll später in agiler Weise hinzugefügt werden.

2. Architektur-Entwurf: Zuschnitt der Komponenten

Für das System sollen Komponenten identifiziert werden, die benötigte Funktionalität realisieren. Ergebnis ist ein Architekturdiagramm, das die Komponenten, ihre Datenflüsse und ihre Schnittstellen darstellt. Im Folgenden muss dieses Diagramm angepasst werden, wenn sich die Architektur ändert, z. B. weitere Komponenten hinzugefügt werden. Wichtige zu diskutierende Fragen sind, ob Komponenten (Services) zustandsbehaftet oder zustandslos sein werden, und ob sie redundant bzw. skalierbar ausgelegt werden sollen. Zudem wird es auch externe Services geben, die unser System verwendet.

3. Definition der Schnittstellen

Die Schnittstellen der identifizierten Komponenten werden konkret festgelegt. Mögliche Stile sind REST aber auch Remote-Procedure-Calls. Hier gilt es, eine gute Art der Dokumentation zu finden und möglichst einheitlich zu verwenden. (Ggf. sinnvolle Systeme sind Swagger/Open-API [2], Protocol Buffers [3] oder Thrift [4], u. a.)

4. Komponenten-Implementierungen — mein PEERREVIEW-System

Programmieren Sie bitte die zugehörigen Komponenten mit ihren Schnittstellen in Isolation. Programmieren Sie bitte auch Test-Clients, die Ihre PEERREVIEW-Komponenten auf verschiedene Weise ansprechen und die ermittelten Daten anzeigen bzw. auf ihre Richtigkeit überprüfen. Sie können auch Test-Systeme verwenden, wie etwa Postman [5].

KI-gestützte Entwicklung: Der Einsatz KI-basierter Werkzeuge (z. B. GitHub Copilot, Claude, ChatGPT o. ä.) zur Unterstützung bei Entwurf, Implementierung, Test und Dokumentation ist ausdrücklich erwünscht. Halten Sie bitte fest, welche Werkzeuge Sie wie eingesetzt haben, welche Ergebnisse hilfreich waren und wo die Grenzen lagen. Diese Reflexion ist Teil der Projektdokumentation.

5. Orchestrierung und Deployment

Die realisierten Komponenten sollen gemeinsam mit Standard-Versatzstücken (Web-Server, Datenbank, Message Broker, ...) zu einem Gesamtsystem orchestriert und auf einem (Internet-)Server bereitgestellt werden.

6. Projekt-Dokumentation

Erstellen Sie bitte eine Dokumentation, in der Sie Ihr Projekt detailliert beschreiben. Dazu gehört die Beschreibung der Architektur, der Schnittstellen, der Orchestrierung, des Deployments sowie eine Reflexion des KI-Einsatzes im Entwicklungsprozess.

Bitte führen Sie die Entwicklung von PEERREVIEW im bereitgestellten Github-Repository MSA-26-PeerReview [6] in geeigneten Branches durch. Definieren Sie auch gerne CI/CD-Pipelines etwa mit GitHub Actions.

Sowohl das PEERREVIEW-System als auch die Test-Clients sollen nach dem Auschecken des Repositories mit Hilfe eines einzigen Kommandos (etwa `make system`, `make client` oder `docker compose up`) übersetzt und dann einfach in Betrieb genommen werden können.

Gegen Ende der Vorlesungszeit stellen Sie bitte Ihre Ergebnisse (Erfahrungen, Schnittstellen, Programme, ...) in einer Präsentation von ca. 30 Minuten plus Vorführung des Systems vor.

Literatur

[1] <https://json.org>

[2] <https://swagger.io/>

[3] <https://developers.google.com/protocol-buffers>

[4] <https://thrift.apache.org/>

[5] <https://www.postman.com/>

[6] <https://github.com/fh-wedel/MSA-26-PeerReview>

[7] <https://v0.dev>